AIOS WORKSHOP — COMPLETE SETUP

Everything You Need. Start to Finish.

Every account, install, prompt, command, and API key. Follow this top to bottom and you'll leave with a fully working AI operating system.

13 skills | 8 commands | ~3 hours to set up | ~$30 NZD/mo ongoing

STEP 0 — BEFORE YOU START

Accounts to Create

Sign up for these before or during setup. Required ones block progress — optional ones add power later.

REQUIRED

Claude Pro

The AI brain that powers everything. Without this, nothing works.

$20 USD/mo (~$34 NZD)

`claude.ai/pro`

REQUIRED

Google Cloud Console

Needed to get your Gmail API credentials so your AI can read and send email.

Free

`console.cloud.google.com`

REQUIRED

Telegram

How you talk to your AI from your phone. Free app, takes 2 minutes to set up a bot.

Free

`telegram.org`

OPTIONAL

Composio

Connects your AI to 250+ tools — Xero, Notion, Slack, HubSpot, Stripe, and more.

Free tier available

`composio.dev`

OPTIONAL

Firecrawl

Web scraping for your AI — research competitors, pull contact info, crawl sites.

Free: 500 scrapes/mo

`firecrawl.dev`

OPTIONAL

Stripe

If you use Stripe for payments — your AI can query invoices, customers, and revenue.

Free (need existing account)

`dashboard.stripe.com`

STEP 1

Install Claude Code

Claude Code is the CLI that runs your AI brain. It needs Node.js first.

1

IF YOU DON'T HAVE IT

Install Node.js

Download and install from nodejs.org. Pick the LTS version. Restart your terminal after installing.

Check if you already have it: run `node --version` in your terminal. If you see a version number, skip this step.

2

IN YOUR TERMINAL

Install Claude Code

terminal

```
$ npm install -g @anthropic-ai/claude-code
```

Verify it worked:

```
$ claude --version
✓ Claude Code v1.x.x
```

3

SIGN IN

Connect to Claude Pro

Run Claude Code for the first time — it will open a browser window to authenticate with your Claude Pro account.

```
$ claude
```

✓ Sign in with the same account as your Claude Pro subscription. This is how the AI runs for free — no API key needed.

STEP 2

Build Your Workspace

Create your AI brain folder, copy the starter kit, and initialise your memory system.

1

IN TERMINAL

Create Your Workspace Folder

```
$ mkdir ~/Desktop/My-AI-Brain
$ cd ~/Desktop/My-AI-Brain
```

2

FILE COPY

Copy the Starter Kit

Copy all the files from your AIOS Starter Kit USB/folder into `~/Desktop/My-AI-Brain`. Keep the folder structure exactly as-is.

3

IN TERMINAL

Clone OpenClaw

OpenClaw is the open-source reference your AI studies to build its memory system.

```
$ git clone https://github.com/openclaw/openclaw.git
```

4

OPEN CLAUDE CODE IN YOUR FOLDER

Launch Claude Code

```
$ claude
```

5

PASTE INTO CLAUDE CODE

Build Your Memory System

PROMPT TO PASTE `/init-memory`

```
1. Go to the OpenClaw repository I just cloned (./openclaw).
2. Study how they built their memory system.
3. Then build me my own version inside my current folder.
4. Use markdown files for storage and SQLite for search.
5. Create Soul.md, User.md, Memory.md, and agent.md.
6. We need a memory folder for daily logs and a memory.db database.
```

6

FILL IN THE TEMPLATES

Personalise Your Brain

Open each file and replace the `[PLACEHOLDER]` sections with your real information.

1. **Soul.md** — How your AI should talk and behave
2. **User.md** — Who you are, your role, your goals
3. **context/business-info.md** — Your business, services, location
4. **context/personal-info.md** — Your responsibilities and preferences
5. **context/strategy.md** — Your current priorities and goals
6. **context/current-data.md** — Your metrics and KPIs

STEP 3

Connect Gmail

Get your Gmail API credentials from Google Cloud so your AI can read, draft, and send email on your behalf.

1

GOOGLE CLOUD CONSOLE

Create a New Project

1. Go to **console.cloud.google.com**
2. Click the project dropdown at the top → **New Project**
3. Name it **My AI Brain** → click **Create**
4. Make sure the new project is selected in the dropdown

2

ENABLE THE API

Turn On Gmail API

1. In the left menu → **APIs & Services** → **Library**
2. Search for **Gmail API** → click it → click **Enable**

3

OAUTH CONSENT SCREEN

Configure Consent Screen

1. Go to **APIs & Services** → **OAuth consent screen**
2. Choose **External** → click **Create**
3. Fill in App name: **My AI Brain**, add your email as support email
4. Click through to **Scopes** → Add scope → search **gmail** → select `gmail.modify` → Save
5. Under **Test users** → add your own Gmail address → Save

4

CREATE CREDENTIALS

Download credentials.json

1. Go to **APIs & Services** → **Credentials**
2. Click **+ Create Credentials** → **OAuth client ID**
3. Application type: **Desktop app** → Name it **My AI Brain** → Create
4. Click **Download JSON** → save the file
5. Rename it to `gmail-credentials.json` and move it to: `~/Desktop/My-AI-Brain/credentials/`

✓ Your `.mcp.json` is already configured to look for the file at `./credentials/gmail-credentials.json` — just make sure the file is in the right place.

5

FIRST RUN

Authorise Access

The first time you use a Gmail command in Claude Code, it will open a browser asking you to authorise access to your Gmail account. Click Allow — this only happens once.

```
# In Claude Code, type:
/inbox
```

STEP 4

Connect Telegram

Talk to your AI from your phone. Two options — pick one. Option A works on any machine. Option B is native to Claude Code.

1

IN TELEGRAM APP

Create Your Bot + Get Your ID

1. Search **@BotFather** → send `/newbot`
2. Give it a name (e.g. **My AI Brain**) and username (e.g. **myaibrain_bot**)
3. Copy your **bot token** — looks like `123456789:ABCdef...`
4. Search **@userinfobot** → send any message → copy your **numeric user ID**

A

OPTION A — PYTHON (WORKS EVERYWHERE)

Build the Bot via Prompt

PROMPT 1 — PASTE INTO CLAUDE CODE `/init-telegram`

```
Build me a Telegram bot adapter for my second brain. It should receive my Telegram messages, read my Soul.md, User.md, Memory.md, and today's log before every reply, then send the message to Claude and return the response to Telegram. Append every exchange to today's log. Only respond to my user ID [YOUR_USER_ID]. My Telegram token is [YOUR_TOKEN].
```

PROMPT 2 — PASTE IMMEDIATELY AFTER

```
Rewrite telegram_bot.py to use subprocess to call the Claude CLI directly instead of any API key. This way it uses my existing Claude subscription. Strip the Claude Code environment variables from the subprocess so there is no nested session conflict.
```

start the bot

```
$ python3 telegram_bot.py

# Keep running in background:
$ nohup python3 telegram_bot.py &

# Stop it:
$ pkill -f telegram_bot.py
```

B

OPTION B — OFFICIAL PLUGIN (NATIVE CLAUDE CODE)

Install Bun + Plugin

MAC / LINUX

```
curl -fsSL https://bun.com/install | bash
```

WINDOWS (POWERSHELL)

```
powershell -c "irm bun.sh/install.ps1|iex"
```

Restart your terminal, then install the plugin:

```
$ claude mcp add-from-github anthropics/claude-plugins-official --subdir telegram
```

Enter your bot token and user ID when prompted. Then restart Claude Code — Telegram is now a native channel.

STEP 5

Add a Heartbeat

Runs every 30 minutes in the background. Checks your memory and today's log. Sends a Telegram message only if something is worth knowing about — silent otherwise.

PASTE INTO CLAUDE CODE `/init-heartbeat`

```
Look at how OpenClaw implemented their heartbeat (./openclaw directory).
Build me a heartbeat.py script that runs every 30 minutes, reads my memory
files and today's log, asks Claude if anything is worth notifying about, then
only sends a Telegram message if yes. Logs everything to heartbeat_debug.log
and prevents repeating notifications within 24 hours using
heartbeat_log.json. Start it automatically when bot.py starts.
```

✓ Files created: **heartbeat.py** (the script), **heartbeat_debug.log** (run log), **heartbeat_log.json** (deduplication — no repeat alerts within 24h)

STEP 6 — OPTIONAL

Run Automatically on Startup

Make your bot and heartbeat start automatically when you log in.

MAC (LAUNCHD)

PASTE INTO CLAUDE CODE

```
Create a macOS launchd plist at ~/Library/LaunchAgents/com.aios-brain.bot.plist that starts telegram_bot.py and heartbeat.py automatically on login. Working directory ~/Desktop/My-AI-Brain. Log to ~/Desktop/My-AI-Brain/logs/.
```

```
$ launchctl load ~/Library/LaunchAgents/com.aios-brain.bot.plist
```

WINDOWS (TASK SCHEDULER)

PASTE INTO CLAUDE CODE

```
Create a Windows Task Scheduler XML that starts telegram_bot.py and heartbeat.py automatically on login. Working directory ~/Desktop/My-AI-Brain.
```

STEP 7 — OPTIONAL

Connect 250+ Tools via Composio

Connect Xero, Notion, Slack, HubSpot, Stripe, GitHub, and hundreds more — all usable directly from Claude Code.

1

SIGN UP + GET KEY

Get Your Composio API Key

1. Go to **composio.dev** → sign up (free)
2. Go to Settings → copy your API key
3. Add to your `.env` file: `COMPOSIO_API_KEY=your-key-here`

2

RUN SETUP

Connect Your Tools

```
$ python3 scripts/setup_client.py
```

Or paste this prompt in Claude Code:

PASTE INTO CLAUDE CODE

```
Set up Composio integrations for my brain. Connect: Gmail, Google
Calendar, Notion, Xero, Stripe. Use COMPOSIO_API_KEY from .env. Show
me what's available after connecting.
```

STEP 8

Launch & Prime

Open Claude Code in your workspace and load all your context with one command.

```
$ cd ~/Desktop/My-AI-Brain
$ claude
```

THEN TYPE THIS COMMAND `/prime`

```
/prime
```

✓ Your AI now knows your business, your goals, your tools, and your context. Start talking to it like a colleague.

REFERENCE

All Commands

Type these directly in Claude Code at any time.

COMMAND	WHAT IT DOES
`/prime`	Load all your context at the start of every session
`/prime-telegram`	Lightweight prime for phone sessions (faster)
`/inbox`	Full Gmail triage — classify emails, draft replies, forward invoices to accounting
`/create-plan`	Design a detailed implementation plan for any project (saves to plans/ folder)
`/implement`	Execute a plan created by /create-plan step by step
`/init-memory`	Bootstrap your memory system from the OpenClaw reference architecture
`/init-telegram`	Set up your Telegram bot — guides you through Option A and Option B
`/init-heartbeat`	Build the proactive heartbeat that notifies you when something needs attention

REFERENCE

All 13 Skills

These activate automatically based on what you say. No commands needed — just talk naturally.

billing-automation
invoice, payment, bill, charge

copywriting
write copy, ad copy, email copy, landing page copy

email-composer
write an email, draft a message, compose

firecrawl-scraper
scrape, crawl, extract from website, research this site

follow-up-nurture
follow up, nurture leads, chase up, pending leads

gmail
check inbox, send email, search emails, gmail

invoice-extractor
extract invoice, read this invoice, invoice data

meeting-notes
meeting notes, summarise this meeting, action items

pdf
create PDF, generate PDF, export as PDF

prompt-engineering
improve this prompt, write a better prompt

skill-creator
create a skill, build a skill, new skill

spreadsheet
spreadsheet, CSV, Excel, Google Sheets, data table

web-designer
build a website, landing page, design a page

REFERENCE

API Keys Checklist

Everything that goes in your `.env` file. Copy from `.env.example` to `.env` and fill in your values.

`TELEGRAM_BOT_TOKEN`	From @BotFather in Telegram	REQUIRED
`ALLOWED_CHAT_IDS`	Your numeric user ID from @userinfobot	REQUIRED
`GMAIL_CREDENTIALS_PATH`	Path to your gmail-credentials.json (./credentials/gmail-credentials.json)	REQUIRED FOR EMAIL
`COMPOSIO_API_KEY`	composio.dev → Settings → API Keys	OPTIONAL
`FIRECRAWL_API_KEY`	firecrawl.dev → Dashboard → API Keys	OPTIONAL
`STRIPE_SECRET_KEY`	dashboard.stripe.com → Developers → API Keys	OPTIONAL
`NOTION_API_KEY`	notion.so/my-integrations → New integration	OPTIONAL

Never share your `.env` file or commit it to GitHub. It contains your private API keys. The `.gitignore` in the starter kit already excludes it.